

Sub-Two-Bit Ternary Weight Storage with Table-Free Arithmetic Decoding for CPU Inference

Justus Theile

Independent researcher

justus.theile@gmail.com

<https://github.com/purpleskull11/bitnet-t5b>

September 2026

Abstract

Ternary (1.58-bit) large language models are stored in practice at two bits per weight, 26 % above the information content of a ternary symbol. We show that the residual redundancy is recoverable on commodity x86 CPUs without lookup tables and without decompressing weights into a wider representation. We introduce **t5b**, a radix-3 code that places five ternary values in one byte at exactly 1.600 bits per weight, and an AVX2 kernel that recovers all five digits from the packed byte using four *independent* `vpmulhbw` instructions per 16-bit view of the byte — eight per 32-byte block — exploiting the fact that a five-trit byte is bounded by 242 and therefore lies well inside the exactness range of the corresponding magic multipliers. The kernel contracts directly on the digits with `vpmaddubsw`, so no weight is ever materialised. We give a roofline argument identifying the condition under which a denser format is profitable — the ratio between a core’s multiply-port throughput and its share of memory bandwidth — and verify it by measuring both quantities directly. Integrated into `llama.cpp` as a distinct `ggml` type alongside the existing two-bit format, t5b reduces the shipped `BitNet-b1.58-2B-4T` checkpoint from 1.10 GiB to 1.00 GiB and yields, under `llama-bench at four threads`, $1.132\times$ prompt and $1.143\times$ generation throughput, faster in five of five invocations. The re-encoding is lossless and the output is bit-identical: perplexity over 20,480 tokens of WikiText-2 agrees with the two-bit baseline to six significant figures. We further report that 4.76 % of the model’s feed-forward neurons are identically zero, that the dead index sets of the gate and up projections coincide in all thirty layers, and that an independent fine-tune revives none of them. All results are from a single AVX2 microarchitecture *without* VNNI; we argue why VNNI is expected to narrow or invert the advantage, and note that an independent system targeting VNNI stores ternary weights at eight bits for that reason.

1 Introduction

Quantisation to ternary weights $w \in \{-1, 0, +1\}$ preserves task performance at the two-billion-parameter scale while removing multiplication from the dominant matrix operation. The resulting models are attractive for CPU deployment: the matrix–vector product requires only sign application and integer accumulation, and the weight matrix is small enough to change which hardware resource binds the computation.

That last point is usually made as a footprint argument and left there. We take it as the design constraint. During autoregressive generation every weight is read exactly once per emitted token and used exactly once, so arithmetic intensity is fixed at one multiply–accumulate per stored weight and the achievable token time is bounded below by

$$t_{\text{token}} \geq \max\left(\frac{N\mu}{\pi}, \frac{N\beta}{8B}\right), \quad (1)$$

where N is the number of weights, β the bits used to store each, μ the instructions issued on the binding execution port per multiply-accumulate, π that port’s throughput, and B the memory bandwidth available to the executing core. The first term is arithmetic; the second is transport.

Existing ternary CPU formats occupy the extremes of the (β, μ) trade-off that (1) describes. `llama.cpp`’s `i2_s` uses $\beta = 2$ with a decode of three shifts and four masks per 128 weights. `bitnet.cpp`’s TL2 and the T-MAC system reach $\beta \approx 1.667$ by table lookup, paying memory for the tables (TL1 is at 2.0); `llama.cpp`’s own TQ1_0 reaches 1.6875 by base-3 packing without a table, and TQ2_0 2.0625. In the opposite direction, *Litespark* stores ternary weights at $\beta = 8$, on the explicit grounds that “using 2-bit packing would require unpacking weights before every computation, negating the performance benefit” — a decision made for targets whose dot-product instruction (VPDPBUSD, ARM SDOT) consumes eight-bit operands.

The base-3 packing itself is not new, and an earlier draft of this paper claimed it was. `llama.cpp` has carried TQ1_0 since August 2024 (pull request #8151): its block structure in `ggml-common.h` is annotated “5 *elements per byte* ($3^5 = 243 < 256$)”, it decodes without a table, and it has an AVX2 dot product. TQ1_0 reaches 1.6875 bits per weight only because each 256-element block carries an f16 scale and a 16-element remainder at two bits; the code bytes themselves are at 1.600. Any claim of novelty for the packing would survive on the arithmetic of that block overhead alone, which is not a claim worth making. We had integrated into this very codebase without checking it, and record that here rather than in a footnote.

What this paper contributes is therefore narrower: a per-tensor scale model that removes the block overhead and is drop-in for the `i2_s` path of `bitnet.cpp`, an interleave identical to `i2_s`’s so the access pattern is unchanged, a decode of four *independent* magic multiplies, a column-blocked matrix-matrix kernel, an explicit and measurable condition for when any of this pays, and an end-to-end evaluation on a real ternary checkpoint against the format that ships with it.

What this is and is not. The mechanism is a *lossless re-encoding* of weights that are already ternary, together with a decode-fused kernel. No weight value changes; the output is bit-identical. It is therefore neither pruning nor weight sharing nor a new quantisation scheme, and it composes with all three.

1.1 Contributions

1. A 1.600-bit variant of the *known* base-3 packing (§3). Five ternary values as the base-3 digits of a byte is TQ1_0’s construction, not ours; ours is the scale model — one per-tensor scale in place of TQ1_0’s per-256-block f16 and its 16-element two-bit remainder, removing the block overhead entirely (1.600 against 1.6875, 20% denser than `i2_s`) — with an interleave identical to `i2_s`’s.
2. A table-free decode in four multiplies (§4). Because a five-trit byte satisfies $x \leq 242$, all four quotients $\lfloor x/3^j \rfloor$ lie inside the exactness range of 16-bit magic multipliers and are obtained *independently* rather than as a serial chain, collapsing a depth-five dependence to depth one.
3. An explicit profitability condition (§5) derived from (1) and verified by measuring both π and B rather than inferring either.
4. Integration and end-to-end evaluation (§7) in `llama.cpp`, with the two-bit path retained as a control and losslessness verified by perplexity agreement over 2.05×10^4 tokens.

5. A structural observation (§8): 4.76 % of the checkpoint’s feed-forward neurons are identically zero, the dead index sets of W_{gate} and W_{up} coincide in all thirty layers, and an independent fine-tune revives none of them.

2 Background

2.1 The information bound

A ternary weight carries at most

$$H_{\text{max}} = \log_2 3 = 1.58496 \text{ bits}, \quad (2)$$

the origin of the “1.58-bit” label, attained only for a uniform distribution. The empirical marginal over all 2,084,044,800 ternary weights of the shipped checkpoint is

$$(p_{-1}, p_0, p_{+1}) = (0.288940, 0.422109, 0.288951), \quad (3)$$

giving a zeroth-order entropy

$$H = - \sum_s p_s \log_2 p_s = 1.5603 \text{ bits}. \quad (4)$$

The excess zero mass *lowers* the floor below $\log_2 3$. Storage at $\beta = 2$ exceeds (4) by 28.2 %; the code of §3 exceeds it by 2.5 %.

2.2 The interface a replacement must preserve

ggml’s ternary kernels do not compute $\sum_i w_i a_i$ directly. They compute on the shifted code $c_i = w_i + 1 \in \{0, 1, 2\}$, the unsigned operand `vpmaddubsw` requires, and the caller removes the shift:

$$\sum_{i=1}^n w_i a_i = \sum_{i=1}^n c_i a_i - \sum_{i=1}^n a_i. \quad (5)$$

The correction depends only on the activations and is therefore evaluated once per matrix–vector product rather than once per row, $O(n)$ against $O(mn)$. Any replacement must expose the same quantity $\sum_i c_i a_i$. Identity (5) requires $c_i \leq 2$; a code $c = 3$ would contribute $3a_i$ while denoting $w = 0$. We verify in §7.1 that code 3 occurs zero times across all 2,084,044,800 weights.

3 The t5b code

3.1 Definition

A block occupies 32 bytes and carries $L = 160$ ternary weights. For byte index $j \in \{0, \dots, 31\}$ and digit index $k \in \{0, \dots, 4\}$,

$$b_j = \sum_{k=0}^4 c(w_{32k+j}) 3^k, \quad c(w) = w + 1 \in \{0, 1, 2\}. \quad (6)$$

The interleave $32k + j$ is `i2_s`’s own layout extended from four digits to five, so digit plane k addresses 32 *consecutive* activations and the memory access pattern is unchanged from the format replaced. The code is injective since $3^5 = 243 \leq 256$, and

$$b_j \leq 2 \sum_{k=0}^4 3^k = 2 \cdot 121 = 242. \quad (7)$$

Density is

$$\beta = \frac{8 \text{ bits}}{5 \text{ weights}} = 1.600, \quad (8)$$

a factor $F = 2/1.6 = 1.25$ fewer bytes than `i2_s`.

3.2 Optimality among aligned radix-3 codes

Packing k trits into $\lceil k \log_2 3 \rceil$ bits gives $\beta(k) = \lceil k \log_2 3 \rceil / k$. Restricting to codes that fit a machine word without straddling:

k	bits	β	fits
4	7	1.750	byte, 1 bit wasted
5	8	1.600	byte, exactly
9	15	1.667	16-bit word, 1 bit wasted
10	16	1.600	16-bit word, exactly

$k = 5$ and $k = 10$ tie and nothing aligned does better. We implement both; §4.3 shows they differ by $1.5\times$ in throughput for reasons of SIMD lane width alone.

3.3 Tail

For n not a multiple of L the final block's unused slots take $c = 1$ ($w = 0$) and the corresponding activations are read from a zero-padded buffer, so both terms of (5) are unaffected. Storage is

$$S(n) = 32 \left\lceil \frac{n}{160} \right\rceil \text{ bytes}, \quad (9)$$

exactly 1.600 bits/weight at $K = 2,560$ and 1.6296 at $K = 6,912$; over the whole checkpoint 1.6076 including tail waste and the 32-byte per-tensor scale record `ggml` appends.

4 Kernel

4.1 Digit recovery without a chain

With $q_j = \lfloor b/3^j \rfloor$ the digits satisfy

$$d_j = q_j - 3q_{j+1}, \quad j = 0, \dots, 4, \quad q_5 = 0. \quad (10)$$

A naive implementation derives q_{j+1} from q_j , a serial dependence of length five. We obtain every q_j directly from b , which (7) makes possible.

Proposition 1 (sufficient range for exact magic division). *Let $D \geq 2$ and let M be any positive integer with $e := MD - 2^{16} \geq 1$. Then*

$$\left\lfloor \frac{xM}{2^{16}} \right\rfloor = \left\lfloor \frac{x}{D} \right\rfloor \quad \text{for every integer } 0 \leq x < \frac{2^{16} - e(D-1)}{e}. \quad (11)$$

Proof. Write $x = qD + r$ with $0 \leq r < D$. Since $MD = 2^{16} + e$,

$$xM = q(2^{16} + e) + rM = q2^{16} + (qe + rM),$$

so $\lfloor xM/2^{16} \rfloor = q + \lfloor (qe + rM)/2^{16} \rfloor$ and it suffices that $qe + rM < 2^{16}$. Using $q \leq x/D$ and $rM \leq (D-1)M = 2^{16} + e - M$,

$$qe + rM \leq \frac{xe}{D} + 2^{16} + e - M,$$

which is below 2^{16} exactly when $xe/D < M - e$. Substituting $M = (2^{16} + e)/D$ gives $xe < 2^{16} - e(D-1)$. $\square$

The bound is sufficient, not tight. Since the operative question is finite — does the identity hold for every $x \leq 242$? — we also determine the exact first-failure point by exhaustive search over the whole 16-bit domain, and it is the exhaustive figure the implementation and its tests pin:

j	$D = 3^j$	M	e	bound (11)	first failure	margin over 242
1	3	21,846	2	$x < 32,766$	$x = 32,768$	$135.4\times$
2	9	7,282	2	$x < 32,760$	$x = 32,768$	$135.4\times$
3	27	2,428	20	$x < 3,251$	$x = 3,293$	$13.6\times$
4	81	811	155	$x < 343$	$x = 485$	$2.00\times$

The tightest case first fails at $x = 485$, exactly twice the largest legal packed byte. All four quotients are therefore exact for every input the code can produce, each costs one `vpmulhuw`, and — crucially — the four are mutually independent, so the depth-five chain collapses to depth one and the multiply port rather than latency becomes the limit. Note $M = 811$ is not the canonical $\lceil 2^{16}/81 \rceil = 810$; the implementation uses the larger constant, which has the smaller range and is still twice what the code can produce. The proposition covers it because its hypothesis is only $MD > 2^{16}$. Digits then follow from (10) using shifts and subtractions ($3q = (q \ll 1) + q$) on the unconstrained integer ports.

4.2 Contraction and the accumulator bound

Digit plane k is a register of 32 bytes in $\{0, 1, 2\}$, exactly the unsigned operand `vpmaddubsw` expects:

$$\text{acc}_{16} += \text{vpmaddubsw}(d_k, a_{32k..32k+31}). \quad (12)$$

A product is at most $2 \times 128 = 256$ — the bound the header and the code use, covering $a = -128$ — and `vpmaddubsw` sums adjacent pairs, so a 16-bit lane grows by at most 512 per plane and $5 \times 512 = 2560$ per block. Folding to 32-bit every Φ blocks is safe iff

$$2560 \Phi \leq 32,767 \iff \Phi \leq 12.80, \quad (13)$$

hence $\Phi = 12$. This is a *worst-case* bound. `llama.cpp`'s own `i2_s` kernel folds every 32 blocks with four planes, up to $128 \times 508 = 65,024$, and is correct only because signed activations cancel; §7.1 shows that removing the cancellation makes it disagree with exact arithmetic in 200 of 200 cases.

4.3 Why bytes and not words

The ten-trits-per-`uint16` code admits a kernel that never forms a digit. Substituting (10) into $\sum_k d_k a_k$ and re-indexing,

$$\sum_{k=0}^{K-1} d_k a_k = q_0 b_0 + \sum_{j=1}^{K-1} q_j b_j, \quad b_0 = a_0, \quad b_j = a_j - 3a_{j-1}, \quad (14)$$

so only the quotient chain and a transformed activation vector are needed, the latter computed once per matrix–vector product. The identity holds modulo 2^{32} rather than term by term: intermediate products are large and cancel.

It is nonetheless slower by $1.5\times$, for reasons of lane width rather than radix. On `uint16` lanes the contraction must use `vpmaddwd`, covering 16 lanes where `vpmaddubsw` covers 32, and the quotient chain competes for the same port. Per 160 weights the word variant issues 19 multiply-port instructions against the byte variant's 13, measuring 30.04 against 33.74 GMAC/s per core (§7).

4.4 Blocking for batched operation

For n_c activation columns, decoding per column costs n_c decodes. Blocking so a decoded plane is contracted against N_C columns held in registers reduces the per-block cost from $N_C(\delta + 5)$ to $\delta + 5N_C$ multiply-port instructions, $\delta = 8$. At $N_C = 8$ this is 48 instructions per 1,280 multiply-accumulates, 26.7 MACs each against i2_s’s 32. Measured at $N_C \in \{2, 4, 8\}$: 50.51, 63.54, 73.26 GMAC/s. We deploy $N_C = 8$; its register pressure (181 stack references per loop) costs less than the amortisation gains.

5 When a denser code pays

Let baseline A and candidate B store the same weights at $\beta_A > \beta_B$ with $\mu_A < \mu_B$ binding-port instructions per multiply-accumulate. Write

$$F = \frac{\beta_A}{\beta_B}, \quad S = \frac{\mu_B}{\mu_A}, \quad (15)$$

and define the *surplus* of the baseline,

$$\Sigma = \frac{\beta_A/(8B)}{\mu_A/\pi} = \frac{\pi\beta_A}{8B\mu_A}. \quad (16)$$

If $\Sigma \leq 1$ the baseline is arithmetic-bound and no $S > 1$ can win. If $\Sigma > 1$ it is transport-bound and, from (1), B is faster iff

$$\boxed{S < \Sigma} \quad (17)$$

in which case it runs a factor $\min(F, \Sigma/S)$ faster. The condition is a property of two hardware rates, not of the code, and both are measurable.

5.1 Measuring π

Eight independent dependence chains, so latency cannot bind, with the clock derived from a dependent add chain retiring one per cycle by construction:

instruction	Gop/s	ops/cycle
vpmaddubsw	4.09	1.05
vpmaddwd	4.10	1.05
vpmulhuw	4.09	1.05
vpmullw	4.08	1.04
vpand	15.43	3.95
vpaddw	11.72	3.00

Measured clock 3.91 GHz. All four multiply-class instructions land on one port at 1.05 per cycle; the rest are three to four times more plentiful. This fixes $\pi = 4.09 \times 10^9 \text{ s}^{-1}$ and identifies the binding resource unambiguously.

Methodological note. An earlier version of this measurement gave all eight chains identical seeds; the compiler proved them equal and kept one, eliminating `vpand` entirely and strength-reducing `vpaddw` into a multiply. It reported 2.98×10^6 and 9.15 ops/cycle against a ceiling near 4. Only the physical impossibility of those figures exposed the defect.

5.2 Measuring B : the surplus is a function of core count

B is a core’s *share*, not the socket’s. Running the baseline kernel at 256 KiB per thread (inside L2) and 64 MiB per thread (four times one CCX’s L3), one thread pinned per physical core:

threads	arithmetic (GMAC/s)	transport (GB/s)	Σ
1	83.70	15.47	1.35
2	162.42	24.94	1.63
3	235.14	33.00	1.78
4	287.54	38.62	1.86
6	471.74	38.86	3.03

Arithmetic scales $5.6\times$ across six cores, per core falling only 6 %. Transport scales $2.5\times$ and is flat from four cores to six. The surplus therefore grows with core count for reasons independent of any kernel. For t5b, $S = 2.43$ measured against $F = 1.25$; by (17) the code should lose at one, two and four threads and win at six, which is what §7.3 observes.

6 Experimental setup

CPU	AMD Ryzen 5 3600 (Zen 2), 6 cores / 12 threads
/proc/cpuinfo	avx2, fma; no AVX-512, no VNNI
Caches	L1d 32 KiB/core, L2 512 KiB/core, L3 32 MiB in two CCX slices
Memory	62 GiB
Compiler	gcc 13.3.0, -O3 -mavx2 -mfma -march=native
Model	microsoft/bitnet-b1.58-2B-4T, 2,084,044,800 ternary weights, 210 tensors
Second model	jpacifico/Aramis-2B-BitNet-b1.58-i2s-GGUF, independent fine-tune
Harnesses	llama-bench, llama-batched-bench, llama-perplexity

The host is shared with an unrelated production workload whose load average rarely falls below 1.8; within-arm spread ranges from 2 % to 146 % with load, comparable to the effect under study. Every comparison is paired with the arm order alternated, and headline figures are replicated across five independent invocations of the upstream benchmark.

7 Results

7.1 Correctness

Unit level. 184,228 assertions across five suites, zero failures; of these the 177,734 covering the format and both kernel variants are in the public repository and run from a clone with a compiler alone (`make test`). The other three suites cover parts of the wider project not published here. Kernels are checked against scalar references derived independently from the specification rather than from the implemented identities, so a shared derivation error cannot pass. Coverage includes exhaustive verification of the four magic-division identities over their whole legal range, deliberate 32-bit wraparound driven 4.96 times past 2^{31} , extreme activations ± 127 , and every block and tail boundary.

Model level, argmax. Nine prompts across two models, greedy decoding, one binary, one seed: identical output by `diff`. A negative control confirms the comparison can fail.

Model level, distribution. Perplexity over 40 chunks of 512 tokens of WikiText-2 (llama.cpp’s own CI copy):

$$\text{PPL}_{\text{i2_s}} = 96.8243 \pm 3.55673, \quad \text{PPL}_{\text{t5b}} = 96.8243 \pm 3.55673.$$

Not only the final estimates but all forty running estimates agree exactly, from [1] 47.2744 to [40] 96.8243; instrumentation confirms the arms differed (51,660 calls through the new path against zero). This compares a floating-point aggregate over 2.05×10^4 forward passes rather than a decoded string.

Code 3. Absent: 0 of 2,084,044,800 weights across all 210 tensors, so (5) holds model-wide.

A defect in the baseline. With activations uniformly at ± 127 , llama.cpp’s i2_s AVX2 kernel disagrees with exact arithmetic in 200 of 200 cases on two of seven tensors, every discrepancy a multiple of 2^{16} — the overflow (13) forbids by construction in our kernel. Whether ordinary activations reach that state is not established; the mechanism is.

7.2 Kernel microbenchmark

One core, weights resident in L2 so transport cannot bind:

kernel	GMAC/s	instr	mul	spills	weights	ceiling	of it
i2_s (upstream)	81.93	21	4	0	128	130.9	63 %
t10 (word)	30.04	50.25	19.25	8.25	160	34.6	87 %
t5b (byte)	33.74	47	13	0	160	50.5	67 %

The ceiling is $\pi \cdot (\text{weights}/\text{mul})$. t5b initially measured 24.36 GMAC/s — slower than t10 despite fewer multiplies — which no end-to-end benchmark could diagnose; disassembly showed 44 stack references in a 425-instruction function. One block per iteration with each plane contracted at the moment it exists gave 47 instructions, zero spills, 33.74 GMAC/s, hence $S = 81.93/33.74 = 2.43$.

7.3 Weight traffic of one token

Replaying one token’s matrix operations over the model’s real shapes and full 940 MB footprint, best of five, arms rotated:

threads	i2_s	t10	t5b	t5b/i2_s
1	34.57 ms	72.73	64.70	0.534
2	20.29	36.84	33.01	0.615
4	13.19	18.82	16.90	0.780
6	15.15	14.49	11.18	1.354

The crossing matches (17) with $S = 2.43$ and the Σ of §5. Across all eight paired six-thread observations the median is $1.12\times$, seven of eight favouring t5b; best against best, noting i2_s does not improve past four threads, is 13.00 ms against 11.71 ms.

7.4 End to end in llama.cpp

	size	pp512 (t/s)	tg128 (t/s)
i2_s	1.10 GiB	119.65 ± 2.27	22.56 ± 1.26
t5b	1.00 GiB	135.45 ± 1.21	25.78 ± 0.87
ratio	0.909	1.132	1.143

Faster in five of five invocations on both measures; per-invocation ratios 1.096, 1.112, 1.119, 1.253, 1.132 (prompt) and 1.099, 1.133, 1.123, 1.242, 1.143 (generation).

The four-thread result contradicted §5’s prediction; the contradiction is now resolved, and the prediction was right. From $S = 2.43$ and $\Sigma_4 = 1.86$, (17) predicts a loss at four threads and the isolated replay of §7.3 delivers one ($0.780\times$); the model gains $1.132\times$. Instrumenting the glue with a cycle counter — `rdtsc` at the invariant TSC rate of 3.599978 GHz, not the core clock, with `llama-bench`’s untimed warmup removed by differencing $r = 2$ against $r = 10$ — gives the matmul time directly for one arm and by difference for the other, since everything else a token does is identical between them:

kernel	in situ	replay	ratio
t5b	16.70 ms/token	16.90 ms	0.988
i2_s	21.18 ms/token	13.19 ms	1.606

The replay predicts *our* kernel to 1.2% and mispredicts `llama.cpp`’s `i2_s` path by 61%. The effective in-situ ratio is $S_{\text{eff}} = 2.43/1.606 = 1.51$ against $\Sigma_4 = 1.86$, so $S < \Sigma$ holds at four threads and (17) predicts the observed gain rather than contradicting it.

What does not survive is the use of the reference `i2_s` kernel as a stand-in for `llama.cpp`’s real performance: it is $1.6\times$ faster than what the model runs, so every ratio computed against it — the 2.43 included — flatters the baseline. No end-to-end number changes. Matmuls are 43.6% of generation wall time, not the 30% estimated from the replay. Why the in-situ path costs $1.6\times$ its reference is not established; the dispatch, the accumulator fold and the per-column post-processing are candidates, and isolating them would mean instrumenting the control arm.

Prompt length. 1.106, 1.152, 1.180, 1.113 at 64, 256, 1024 and 2048 tokens: the advantage does not decay with length. We had predicted decay on the grounds that a prompt pass is arithmetic-bound; at these shapes the weights still stream from memory, so the byte saving remains live and the prediction was wrong in its premise.

Concurrency. Total throughput at 1, 2, 4, 8 parallel sequences: 1.041, 1.041, 1.027, 1.097. The advantage *grows*: N concurrent sequences turn generation into a matrix–matrix product with N columns, across which the decode amortises.

Across checkpoints. On the independent fine-tune: identical output on three prompts, $1.120\times$ prompt and $1.075\times$ generation.

Across depth. Repeating the block stack yields a 4.50 B-parameter, 60-layer graph `llama.cpp` loads and runs with real attention and KV cache: $1.165\times$ and $1.129\times$, both margins clear of their error bars. That artefact is *not a model* — the same thirty blocks run twice — and no quality claim is made from it.

7.5 Two ratios a reader will ask about immediately

Why 25 % fewer weight bytes give 14 % more throughput. At 21.9 tokens per second a token takes 45.7 ms, and the replay of §7.3 puts a token’s `i2_s` matmuls at about 14 ms. Weight streaming is therefore roughly 30 % of a token; the rest is attention, the KV cache, RoPE, the norms, the activation quantisation, thread barriers and graph overhead, all identical between the arms. Read as bandwidth, the generation loop uses 11.4 GBs^{-1} of the 38.9 this machine delivers: it is not at the memory wall.

Why the file shrinks by only 9 %. The ternary tensors are 521.0 MB of a 1,187.8 MB file. One f16 tensor — the 128k-vocabulary embedding — is 656.7 MB, 55.3 % of the file, and does not change. The 102.2 MB saved is 19.6 % of what the format touches and 8.6 % of the file. The ternary share, and therefore the file saving, rises with model depth at fixed vocabulary.

7.6 A negative result on row tiling

Tiling four weight rows against one activation vector measures $1.19\times$ in cache and $0.52\times$ from memory. A tile of R rows by C columns loads each weight once and spends it on RC contractions; a matrix–vector product has $C = 1$, so tiling rows buys no reuse and merely reorders loads. Upstream draws the same line structurally: its `ggml_gemv_i2_i8_s` does not tile, while the 4×4 tile lives in `ggml_gemm_i2_i8_s`.

8 Dead feed-forward neurons

2.9036 % of all weight-matrix rows are identically zero — an output neuron that cannot fire for any input.

tensor	dead rows	share
W_{gate}	9,870 / 207,360	4.76 %
W_{up}	9,870 / 207,360	4.76 %
W_v	103 / 19,200	0.54 %
W_q	4 / 76,800	0.01 %
$W_{\text{down}}, W_k, W_o$	0	0.00 %

They concentrate at the front: 43.59 % of layer 1’s feed-forward neurons, 27.58 % of layer 2’s, 14.77 % of layer 3’s, near zero by layer 8. **In all thirty layers the dead index set of W_{gate} equals that of W_{up}** — not the same cardinality, the same indices. In a SwiGLU block, $h = W_{\text{down}}(\sigma(W_{\text{gate}}x) \odot W_{\text{up}}x)$, the two are combined element-wise, so a zero gate row renders the matching up row irrelevant and conversely; training removed them in pairs. W_{down} has none, the same fact along the other axis: there a dead neuron is a dead *column*.

They survive fine-tuning exactly: against the independent fine-tune, 210 of 210 tensors have identical dead index sets, all 19,847 rows, 100.00 %. Whatever removed them occurred in pre-training.

Skipping the rows saves 2.4380 % of weight bytes *and* the same share of multiply–accumulates. Removing the neurons outright would save 3.64 % — a dead neuron also renders its W_{down} column useless, $9,870 \times 7,680$ weights — at zero cost to the element-wise product, but *not* provably zero cost to the layer: BitNet places a sub-layer RMS norm between the product and W_{down} , and an RMS norm divides by the root mean over all 6,912 entries, so deleting zeros changes the denominator. Skipping the rows in the kernel is exact; removing them is exact only with that divisor pinned to the original width. The route is *not* blocked by the file format, which this work asserted for some hours without checking: `llama.cpp` reads `LLM_KV_FEED_FORWARD_LENGTH`

through `get_key_or_arr` into a per-layer array (`llama-model.cpp:1117`), so per-layer feed-forward widths are a first-class concept and the removal needs no format change and no kernel work. It is also *unconditionally* exact on the product, precisely because the dead index sets coincide: with up_i zero as well, the neuron contributes nothing for any activation.

9 Limitations

The port analysis is Zen 2 specific before VNNI even enters. §5 measures one multiply-class port at 1.05 instructions per cycle against 3.0 to 4.0 for the cheap classes. A reviewer ran the same benchmark on an Intel part with AVX-512 and obtained 1.60 against 3.09: two 256-bit multiplier ports rather than one. The ratio (17) turns on is therefore a property of the microarchitecture at a finer grain than the instruction set, and the surplus table would have to be re-measured on any other part before (17) could be applied to it.

One microarchitecture, and the decisive property is absent from it. All results are from a single Zen 2 part reporting `avx2` and `fma` and nothing else. `VPDPBUSD` collapses the contraction into one instruction while leaving the decode unchanged; by (16) this raises S and lowers Σ , moving (17) against a packed code. The *Litespark* system, targeting VNNI and ARM SDOT, stores ternary weights at eight bits for precisely this reason. **The central result should be read as a property of AVX2 without VNNI until it is run on a VNNI part.** The code compiles for `-mavxvnni` here but cannot execute, and no such measurement is claimed.

One model size and one architecture family. No public ternary checkpoint larger than 2B exists. The synthetic 60-layer graph doubles weight traffic but not head count, hidden size or vocabulary.

Measurement environment. The host is shared and run-to-run spread is comparable to the effect. An earlier draft reported “four runs out of four” at $1.165\times$; those four were taken in the quietest window available, and four further runs at ordinary load include one the denser code loses. The headline is therefore the five-invocation replication.

Verification scope. The round trip is exact on all 210 tensors and all 2,084,044,800 weights; what is limited to seven tensors is the dot-product comparison and the read-back from disk.

Task benchmarks. None. The format question does not admit one — output is bit-identical — and where this *model* stands against others is a different question this paper does not address. We ran a cross-model perplexity comparison and do not report it as a result: the checkpoint’s GGUF requires `tokenizer.ggml.pre` and the EOS id to be supplied through `--override-kv` because its own metadata is wrong, so an absolute perplexity from it is at least as likely to reflect a tokenisation artefact as the model. It does not affect the identity result, where both arms are affected equally.

Engineering. The blocked matrix–matrix kernel spills 181 times per loop at its chosen width; narrower widths spill less and measure slower. The type identifier used (43) is not reserved by upstream, so a stock `llama.cpp` cannot read the file by construction rather than by accident.

10 Conclusion

The gap between the 1.58 bits a ternary weight carries and the 2.00 it is stored in is recoverable on commodity AVX2 hardware without lookup tables. Five ternary values fit a byte at exactly

1.600 bits; all four radix-3 quotients are extractable from that byte with one independent multiply each, because the byte is bounded by 242 and lies deep inside the exactness range of 16-bit magic multipliers; and the contraction proceeds on the digits, so no weight is materialised.

Whether the resulting 25 % reduction in weight traffic is profitable is not a property of the code. It is decided by (17), a comparison between a core’s multiply-port throughput and its share of memory bandwidth, both measurable in minutes. On the part measured here the condition holds from four cores upward, and the deployed model is 9 % smaller and 13 % to 14 % faster with bit-identical output.

The same inequality predicts the result will narrow on a part with VNNI, and an independent system built for such parts chose the opposite extreme of the same trade-off. That is not a caveat appended to a positive result; it is the result, which is that weight density is a hardware-dependent optimisation with a computable decision rule rather than a property of the model.

Data and code availability

The format, the kernels, the benchmarks, the converters and every evidence file cited here are public under the Apache License 2.0 at

<https://github.com/purpleskull11/bitnet-t5b>

with the layout below. No model weights are distributed and no upstream source is redistributed verbatim; the integration is applied as a patch to a checkout the user supplies, and the files that reproduce or adapt an upstream contract say so in their own headers and in NOTICE.

claim	file
code histogram, code 3 absence, entropy	results/tensor_structure.txt, results/real_weights_che
instruction-port throughput π	benchmarks/bench_ports.c
surplus Σ against core count	results/thread_headroom.txt, benchmarks/bench_threads.
kernel rates, instruction and spill counts	results/weight_density.txt, benchmarks/bench_alu.c
weight traffic of one token	benchmarks/bench_token.c
end-to-end throughput	results/inference_t5b.txt
perplexity agreement between the formats	results/perplexity_t5b.txt
round trip on real weights	results/real_weights_check.txt
dead feed-forward neurons	results/dead_neurons.txt, tools/dead_neurons.py

The format and its kernels are `src/ternary_t5b.{c,h}` with the `ggml`-facing contract in `src/ggml_t5b_glue.{c,h}` and the suite in `src/test_t5b.c`; the word-lane variant of §4.3 is `src/ternary_t10.{c,h}`; the converter is `tools/gguf_to_t5b.py`.

What the public repository does not contain. The measurements that need a model — §7’s end-to-end table, the perplexity agreement — additionally require a `llama.cpp` build carrying the type, a converted GGUF and the checkpoint itself. The integration is supplied as a patch against a named upstream commit rather than as copied sources, and the `i2_s` reference kernel used as the baseline is fetched from that commit by a script rather than redistributed. Everything else — the packing, the kernels, the suite, all four benchmarks, the converters and every evidence file — builds and runs from a clone with a compiler and Python alone.

References

- [1] S. Ma, H. Wang, L. Ma, L. Wang, W. Wu, P. Dong, L. Zhang, J. Xue, F. Wei. *The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits*. arXiv:2402.17764, 2024.

- [2] Microsoft. *BitNet b1.58 2B4T Technical Report*. arXiv:2504.12285, 2025.
- [3] J. Wang, H. Zhou, T. Song, S. Cao, Y. Xia, T. Cao, J. Wei, S. Ma, H. Wang, F. Wei. *Bitnet.cpp: Efficient Edge Inference for Ternary LLMs*. arXiv:2502.11880, 2025.
- [4] J. Wang, H. Zhou, T. Song, S. Mao, S. Ma, H. Wang, Y. Xia, F. Wei. *1-bit AI Infra: Part 1.1, Fast and Lossless BitNet b1.58 Inference on CPUs*. arXiv:2410.16144, 2024.
- [5] J. Wei, S. Cao, T. Cao, L. Ma, L. Wang, Y. Zhang, M. Yang. *T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge*. EuroSys, 2025. arXiv:2407.00088.
- [6] compilade. *ggml-quants: ternary packing for TriLMs and BitNet b1.58*. llama.cpp pull request #8151, August 2024.
- [7] *Litespark Inference on Consumer CPUs: Custom SIMD Kernels for Ternary Neural Networks*. arXiv:2605.06485.
- [8] S. Han, H. Mao, W. J. Dally. *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. ICLR, 2016. arXiv:1510.00149.
- [9] T. Zhang et al. *70% Size, 100% Accuracy: Lossless LLM Compression for Efficient GPU Inference via Dynamic-Length Float*. arXiv:2504.11651, 2025.
- [10] G. Gerganov et al. *llama.cpp / ggml*.
- [11] S. Merity, C. Xiong, J. Bradbury, R. Socher. *Pointer Sentinel Mixture Models*. arXiv:1609.07843, 2016.